

Lab_Assignment_1.3

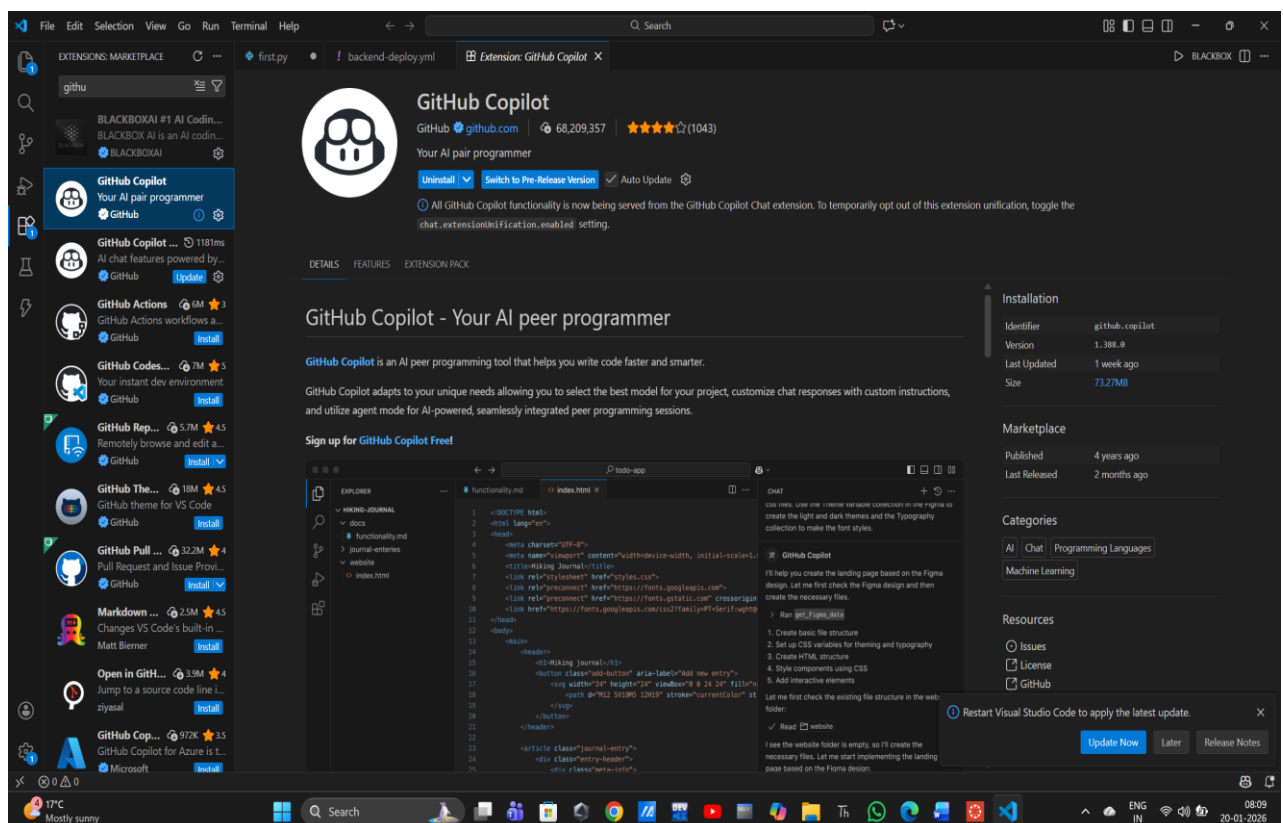
AI Assisted Coding B37 2303A52077

Lab 1: Environment Setup – GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow

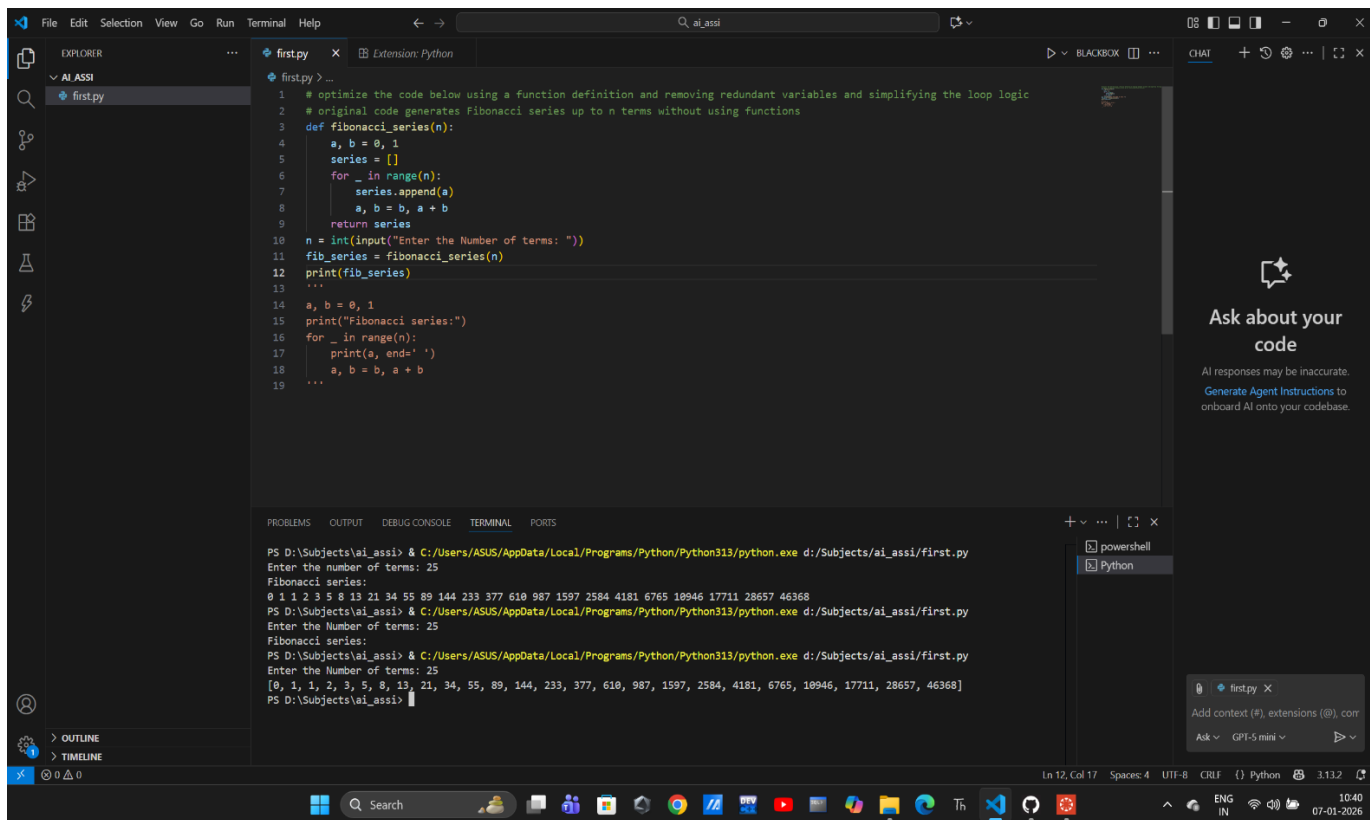
Lab Objectives:

- To install and configure GitHub Copilot in Visual Studio Code.
- To explore AI-assisted code generation using GitHub Copilot.
- To analyze the accuracy and effectiveness of Copilot's code suggestions.
- To understand prompt-based programming using comments and code context

Task 0 : Install and configure GitHub Copilot in VS Code



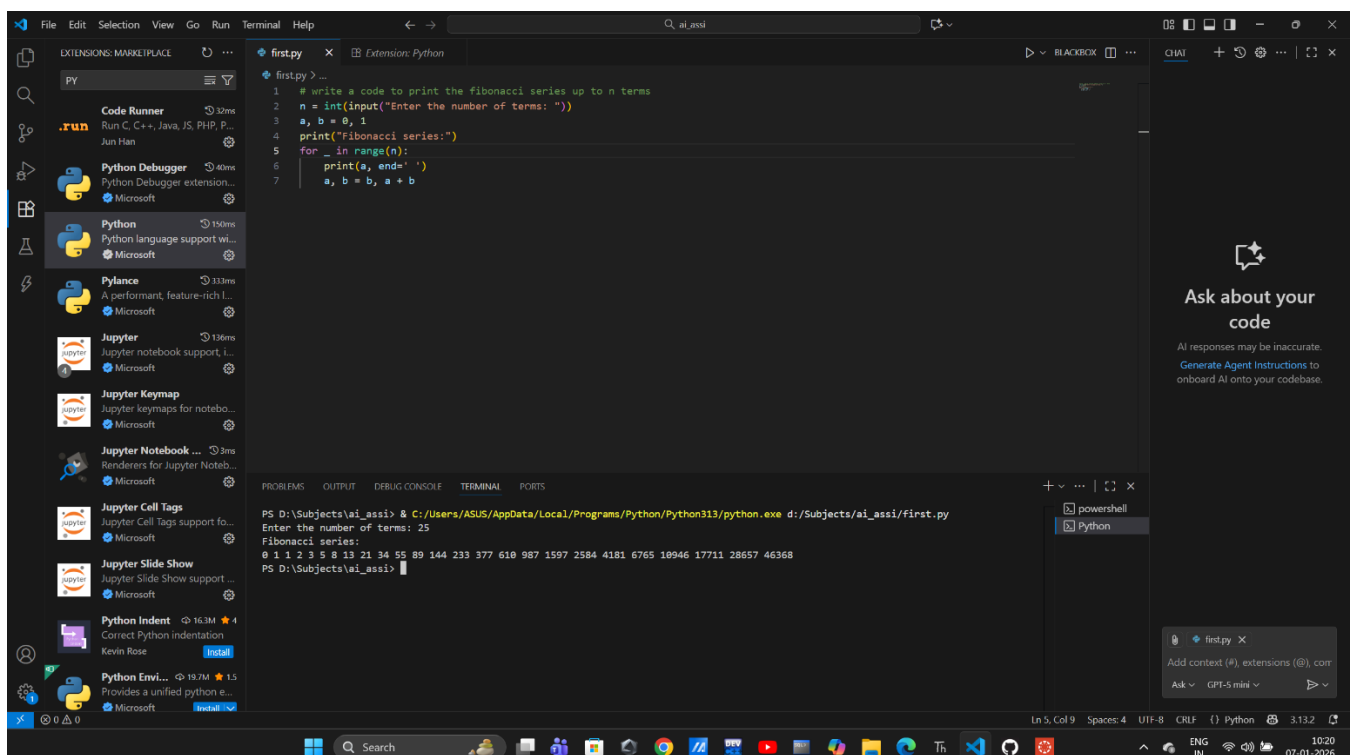
Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)



```
1 # optimize the code below using a function definition and removing redundant variables and simplifying the loop logic
2 # original code generates Fibonacci series up to n terms without using functions
3 def fibonacci_series(n):
4     a, b = 0, 1
5     series = []
6     for _ in range(n):
7         series.append(a)
8         a, b = b, a + b
9     return series
10 n = int(input("Enter the Number of terms: "))
11 fib_series = fibonacci_series(n)
12 print(fib_series)
13 ...
14 a, b = 0, 1
15 print("Fibonacci series:")
16 for _ in range(n):
17     print(a, end=' ')
18     a, b = b, a + b
19 ...
```

PS D:\Subjects\ai_assi> C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe d:/Subjects/ai_assi/first.py
Enter the number of terms: 25
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368
PS D:\Subjects\ai_assi> C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe d:/Subjects/ai_assi/first.py
Enter the Number of terms: 25
Fibonacci series:
PS D:\Subjects\ai_assi> C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe d:/Subjects/ai_assi/first.py
Enter the Number of terms: 25
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368]
PS D:\Subjects\ai_assi>

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

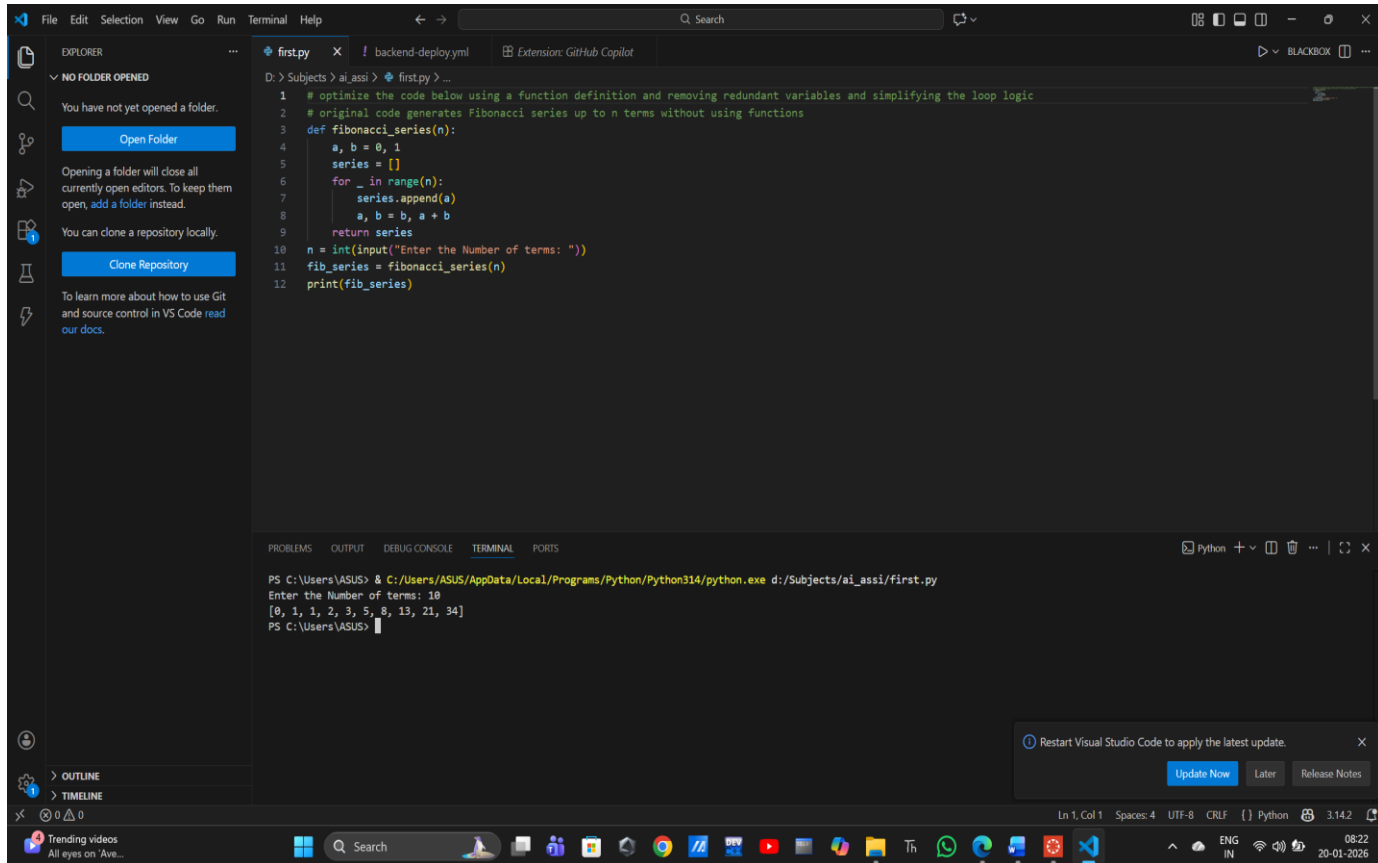


```
1 # write a code to print the fibonacci series up to n terms
2 n = int(input("Enter the number of terms: "))
3 a, b = 0, 1
4 print("Fibonacci series:")
5 for _ in range(n):
6     print(a, end=' ')
7     a, b = b, a + b
```

PS D:\Subjects\ai_assi> C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe d:/Subjects/ai_assi/first.py
Enter the number of terms: 25
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368
PS D:\Subjects\ai_assi>

The code was optimal at task 1 itself

Task 3: *Modular Design Using AI Assistance (Fibonacci Using Functions)*



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a file named 'first.py'. The main editor displays the following Python code:

```
1 # optimize the code below using a function definition and removing redundant variables and simplifying the loop logic
2 # original code generates Fibonacci series up to n terms without using functions
3 def fibonacci_series(n):
4     a, b = 0, 1
5     series = []
6     for _ in range(n):
7         series.append(a)
8         a, b = b, a + b
9     return series
10 n = int(input("Enter the Number of terms: "))
11 fib_series = fibonacci_series(n)
12 print(fib_series)
```

The TERMINAL panel at the bottom shows the execution of the script:

```
PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe d:/Subjects/ai_assi/first.py
Enter the Number of terms: 10
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\ASUS>
```

Sample i/p & o/p :

Enter the Number of terms: 10

[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Task 4: *Comparative Analysis - Procedural vs Modular Fibonacci Code*

Code 1: Procedural Fibonacci Code (Task 1)

$a, b = 0, 1$

```

print("Fibonacci series:")
for _ in range(n):
    print(a, end=' ')
    a, b = b, a + b

```

Code 2: Modular Fibonacci Code (Task 3)

```

def fibonacci_series(n):
    a, b = 0, 1
    series = []
    for _ in range(n):
        series.append(a)
        a, b = b, a + b
    return series

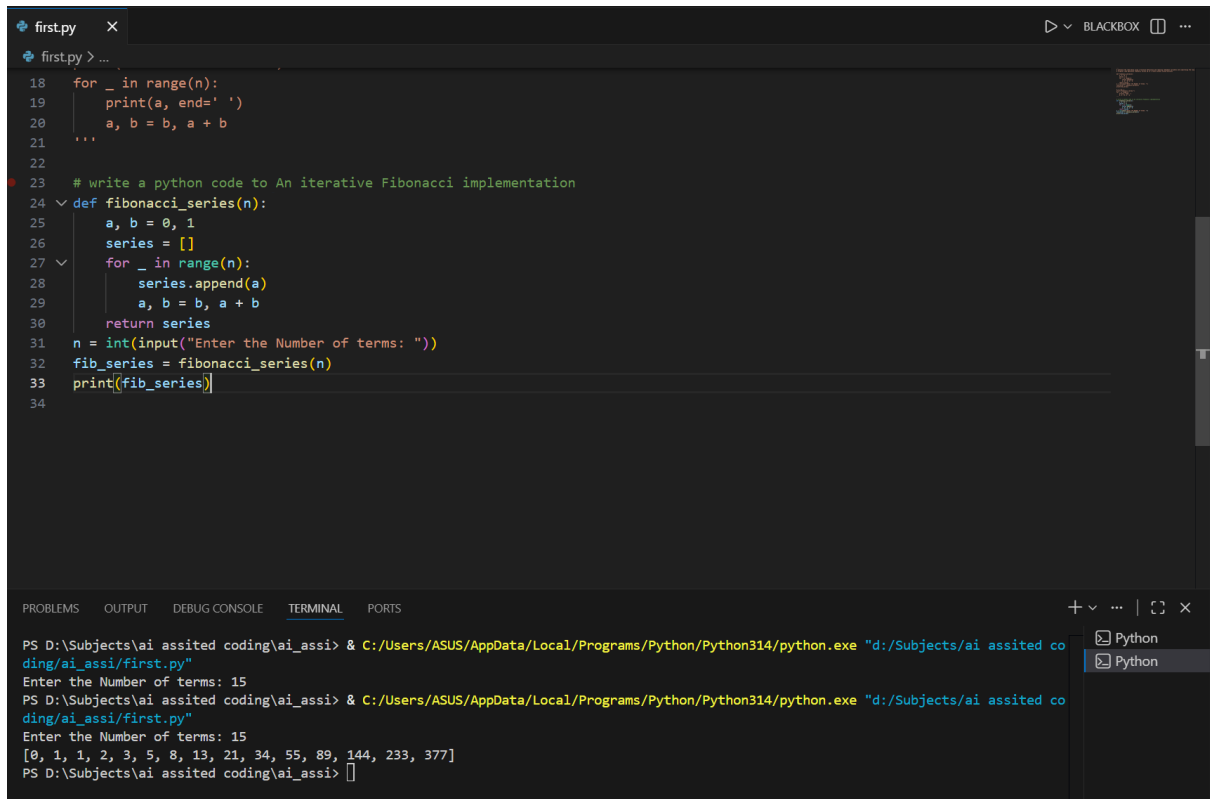
n = int(input("Enter the Number of terms: "))
fib_series = fibonacci_series(n)
print(fib_series)

```

Criteria	Code 1	Code 2
Code Clarity	Moderate	High
Reusability	Low	High
Debugging Ease	Low	High
Larger System Use	Not Suitable	Highly Suitable

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

An iterative Fibonacci implementation



The screenshot shows a code editor with a file named `first.py`. The code defines a function `fibonacci_series(n)` that generates the first `n` Fibonacci numbers iteratively. It uses a loop to calculate each number and append it to a list. The main program prompts the user for the number of terms and prints the resulting list.

```
18 for _ in range(n):
19     print(a, end=' ')
20     a, b = b, a + b
21 ...
22
23 # write a python code to An iterative Fibonacci implementation
24 def fibonacci_series(n):
25     a, b = 0, 1
26     series = []
27     for _ in range(n):
28         series.append(a)
29         a, b = b, a + b
30     return series
31 n = int(input("Enter the Number of terms: "))
32 fib_series = fibonacci_series(n)
33 print(fib_series)
34
```

The terminal output shows the execution of the script. The user enters 15, and the program outputs the first 15 Fibonacci numbers: `[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]`.

Execution Flow

1. *Function is defined*
2. *User input is taken*
3. *Function is called*
4. *Fibonacci numbers are calculated*
5. *Result is returned*
6. *Output is displayed*

A recursive Fibonacci implementation

```

35 #write a python code to An recursive Fibonacci implementation
36 def fibonacci_recursive(n):
37     if n <= 0:
38         return []
39     elif n == 1:
40         return [0]
41     elif n == 2:
42         return [0, 1]
43     else:
44         series = fibonacci_recursive(n - 1)
45         series.append(series[-1] + series[-2])
46         return series
47 n = int(input("Enter the Number of terms: "))
48 fib_series_recursive = fibonacci_recursive(n)
49 print(fib_series_recursive)
50

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe "d:/Subjects/ai assisted co
ding/ai_assi/first.py"
Enter the Number of terms: 15
PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe "d:/Subjects/ai assisted co
ding/ai_assi/first.py"
Enter the Number of terms: 15
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
PS D:\Subjects\ai assisted coding\ai_assi>

```

Execution Flow

1. Function is defined
2. User input is taken
3. Recursive function calls itself until base case
4. Results are built during backtracking
5. Final list is returned
6. Output is printed

Aspect	Iterative Fibonacci	Recursive Fibonacci
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(1)$	$O(n)$
Speed	Fast	Slower
Stack Overflow Risk	No	Yes
Suitable for Large n	☑ Yes	✗ No
Code Simplicity	Moderate	High
Recommended	☑ Best Choice	✗ Avoid